

DATA SCIENCE AND MACHINE LEARNING

Day 12

Dhiraj Bashyal
AI/ML Engineer

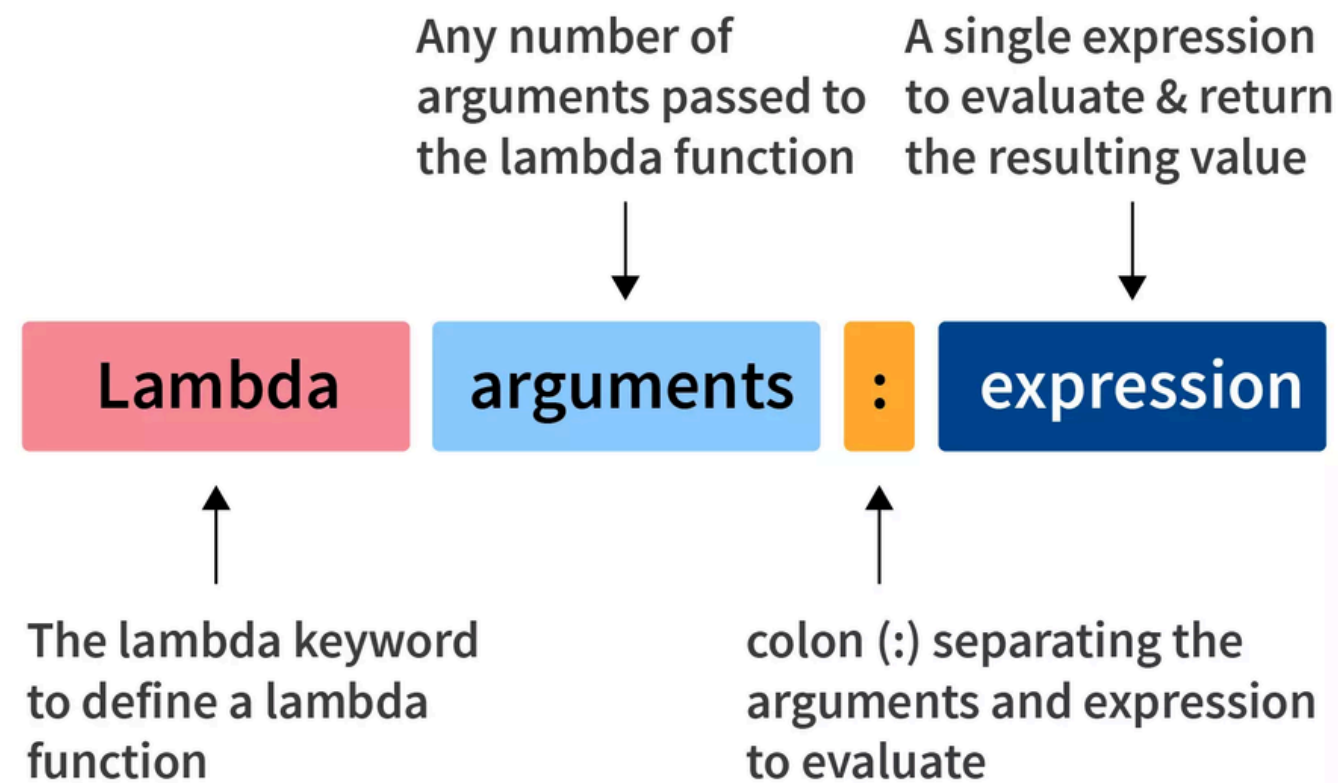
Topics

Day 12

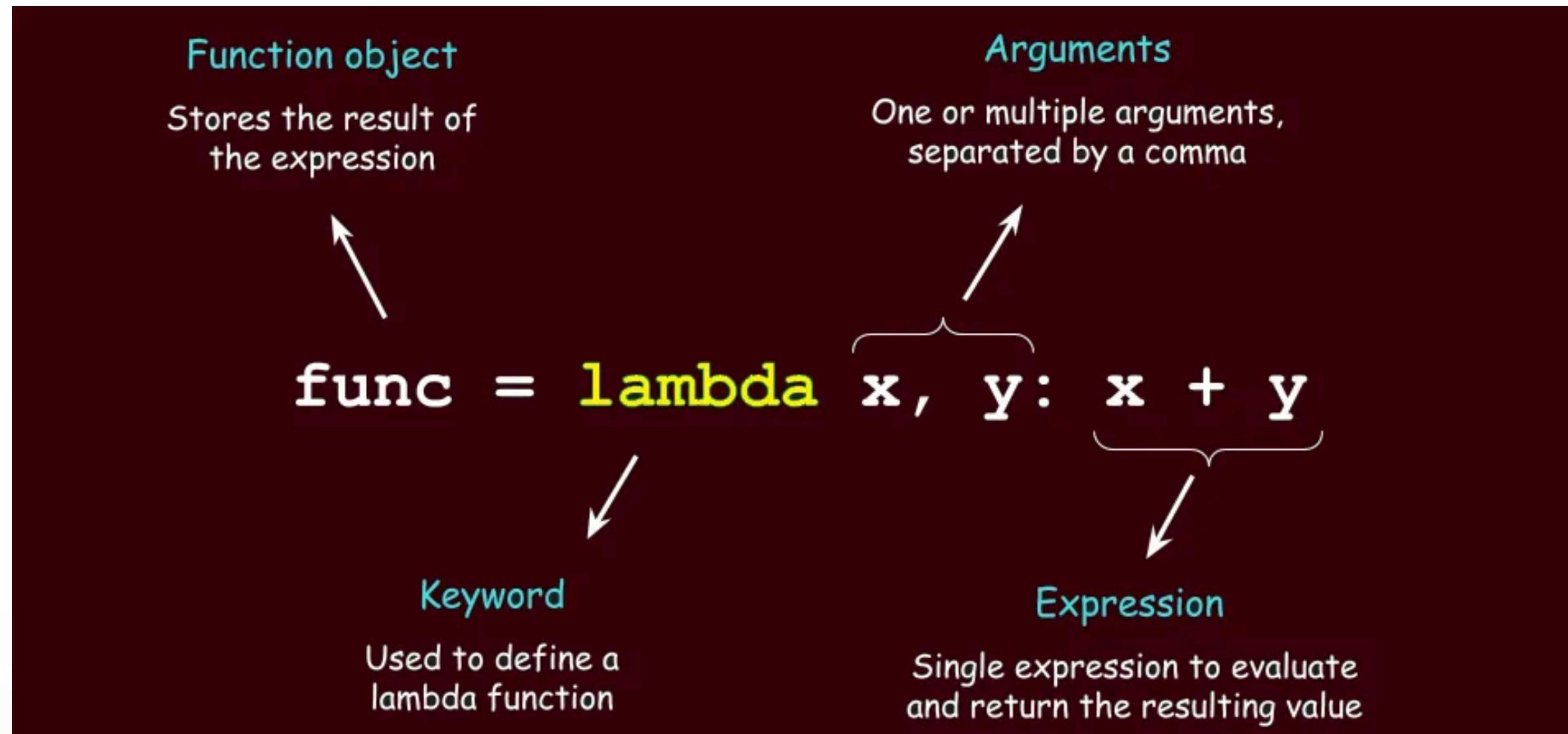
- **lambda function**
- **List & dictionary comprehensions.**

Python lambda function

- Lambda functions are a special kind of function in Python. Unlike the usual functions that we define using the def keyword, lambda functions are small, anonymous functions that we define using the lambda keyword.
- "Anonymous" means that these functions don't have a name. They are quick and simple, and we use them for short tasks.



Python lambda function



Python lambda function

```
def square(x):  
    return x ** 2  
square_lambda = lambda x: x ** 2  
print(square_lambda(5))
```

Using with **filter()**:

```
numbers = [1, 2, 3, 4, 5, 6]  
evens = list(filter(lambda x: x % 2 == 0, numbers))  
print(evens)
```

Python lambda function

Sort a List of Tuples

```
students = [  
    ("Ram", 80),  
    ("Hari", 95),  
    ("Sita", 85)  
]
```

```
students.sort(key=lambda x: x[1])  
#list.sort(reverse=True|False, key=myFunc)  
  
print(students)
```

Python lambda function

Using map() : Convert numbers to squares.

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(map(lambda x: x ** 2, numbers))
```

```
print(result)
```

Python lambda function

Using filter() : Get even numbers only.

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(result)
```

Python lambda function

Using reduce() : Sum all numbers.

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
```

```
result = reduce(lambda a, b: a + b, numbers)
```

```
print(result)
```

Python lambda function

Sort Dictionary List

```
employees = [  
    {"name": "Ram", "salary": 50000},  
    {"name": "Hari", "salary": 30000},  
    {"name": "Sita", "salary": 70000}  
]  
  
employees.sort(key=lambda emp: emp["salary"])  
  
print(employees)
```

Are Lambda Functions Faster in Python?

Python lambda function

No, lambda functions are not faster than regular functions in Python

When to Use Lambda vs. Def

Situation

One-line temporary function

Sorting key

`map()`, `filter()`, `reduce()`

Complex logic

Reusable function

Multiple statements

Documentation needed

Use

`lambda`

`lambda`

`lambda`

`def`

`def`

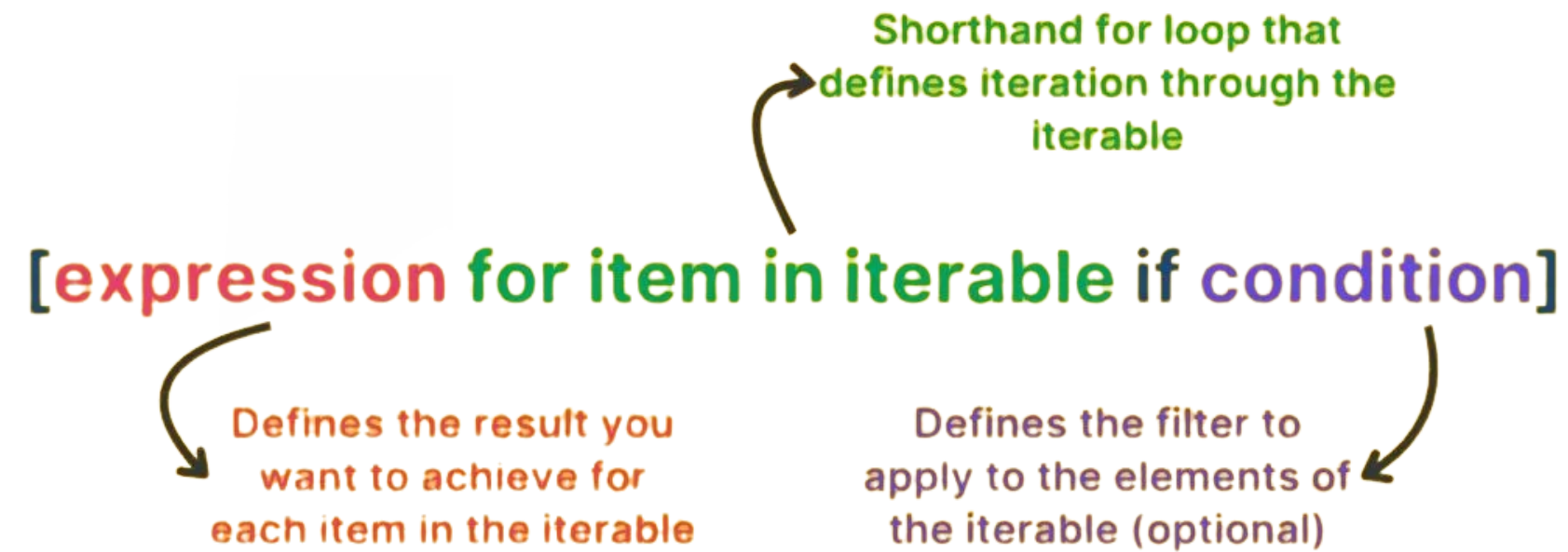
`def`

`def`

Interview Answer:

Use lambda for small, anonymous, one-line functions that are used temporarily, especially with functions like `map()`, `filter()`, and `sorting`. Use `def` when the function contains multiple statements, complex logic, needs documentation, or will be reused throughout the program.

Python List Comprehension



List comprehension is a compact way to create lists in Python and manipulate/modify them. It's a single line of code that replaces the need for a traditional loop when generating or transforming lists.

- It combines the logic of loops with the elegance of Python's functional programming features, making it a go-to for efficient list creation and transformation.
- In simpler terms, list comprehension allows you to write a loop and an expression in one line, making your code more compact and often easier to read.

Python List Comprehension

[expression for item in iterable if condition]

```
Unstop = ["welcome", "to", "dsml", "class"]  
result = [word.upper() for word in Unstop if 'e' in word]  
print(result)
```

Python List Comprehension

Normal :

[expression for item in iterable]

With Condition

[expression for item in iterable if condition]

With If-Else

[true_value if condition else false_value for item in iterable]

Python List Comprehension

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = [num ** 2 for num in numbers]
```

```
print(squares)
```

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
evens = [num for num in numbers if num % 2 == 0]
```

```
print(evens)
```

Python List Comprehension

```
numbers = [1, 2, 3, 4]
```

```
result = ["Even" if num % 2 == 0 else "Odd" for num in numbers]
```

```
print(result)
```

```
list1 = [1, 2]
```

```
list2 = ['A', 'B']
```

```
pairs = [(x, y) for x in list1 for y in list2]
```

```
print(pairs)
```

Python Dictionary Comprehension

The minimal syntax for dictionary comprehension is:

dictionary = {key: value for vars in iterable}

```
{ key: value for vars in iterable }  
|      |      |      |      |  
{ num: num*num for num in range(1, 11) }
```

Dictionary Comprehension with Condition

Create a dictionary containing only even numbers.

{key: value for item in iterable if condition}

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = {num: num**2 for num in numbers if num % 2 == 0}
```

```
print(result)
```

If Conditional Dictionary Comprehension

```
original_dict = {'ram': 38, 'sita': 48, 'gita': 57, 'john': 33}
```

```
even_dict = {k: v for (k, v) in original_dict.items() if v % 2 == 0}  
print(even_dict)
```

Multiple if Conditions in Dictionary Comprehension

```
original_dict = {'Dhiraj': 38, 'Ram': 48, 'sita': 57, 'john': 33}
```

```
new_dict = {k: v for (k, v) in original_dict.items() if v % 2 != 0 if v < 40}  
print(new_dict)
```

if-else Conditional Dictionary Comprehension

```
{  
key: value_if_true if condition else value_if_false  
for item in iterable  
}
```

```
numbers = [1, 2, 3, 4, 5]
```

```
result = {  
    num: "Even" if num % 2 == 0 else "Odd"  
    for num in numbers  
}
```

```
print(result)
```

Practice Questions

- Write a program using list comprehension to convert a list of names into uppercase.
- Create a dictionary using dictionary comprehension where each word in a list is a key and its length is the value.
- Use a lambda function with filter() to extract all odd numbers from a list.
- Use a lambda function with map() to add 10 to every element in a list.
- Given a list of student names, use list comprehension to select names with more than 4 characters and dictionary comprehension to store each selected name with its character count.

